

Introduction to vLLM's new Rust Frontend

ZHAO Ziqi

Member of Technical Staff @ **Inferact**

PyTorch Meetup Singapore | May 2026

Introduction

vLLM

- vLLM is the most popular open-source inference engine
- Aims for high-throughput, low-latency LLM serving
- Features: paged attention, continuous batching, prefix caching, different parallelism strategies

Inferact

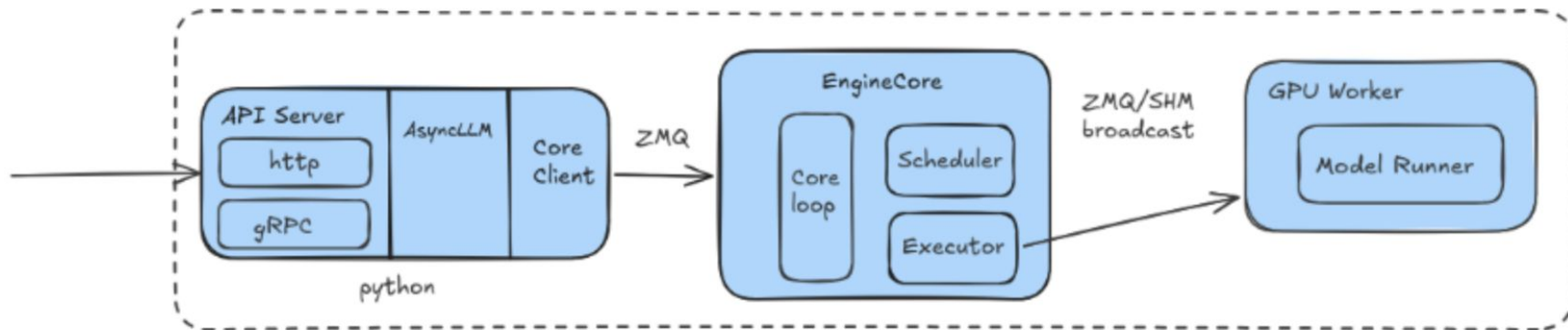
- The company behind vLLM
- Founded by original creators and core maintainers of vLLM

This work explores a Rust frontend (API server) while keeping the existing vLLM engine in Python.

Current Serving Architecture

- Frontend exposes OpenAI protocol as HTTP endpoints
- Frontend transforms requests into lower-level **EngineCoreRequest**
- Engine executes the model, produces tokens in **EngineCoreOutput**
- Frontend transforms back to API responses

ZMQ is the boundary between frontend and backend.



What The Frontend Owns

- Public API surface
 - API schema, validation, errors, usage, streaming outputs
- Input processing
 - Model-specific configuration
 - Chat template & tokenization
 - Multimodal preprocessing
- Output processing
 - Detokenization: stop strings, logprobs
 - Reasoning & tool call parsing
- Operational behavior
 - Health checks, metrics, request lifecycle, graceful shutdown
 - Internal load balancing under DP deployment

Motivation: Performance

GPUs are getting faster, so CPU-side overhead becomes more visible.

- Python limitations
 - GIL
 - dynamic typing
 - GC
- Rust advantages
 - More efficient
 - High concurrency within single process
 - Predictable memory management

Give frontend more headroom as the serving layer grows.

Motivation: Complexity

Python frontend grows organically, accumulating complexity.

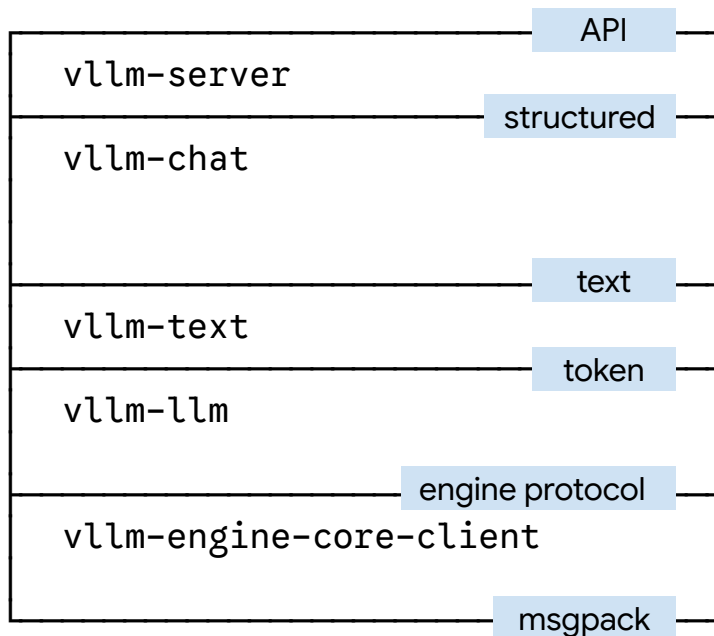
- Necessary
 - Model-specific code for wide model support
 - Useful extensions to OpenAI protocol
- Technical debt
 - Backward compatibility for leaked internal API
- Working around Python performance limits
 - Multi-processing for frontend concurrency
 - Complex coordination

Rust frontend is a chance to redraw the boundaries.

Motivation: Accessibility → Reliability

- At the first stage, Python is natural & obvious
 - Best ML ecosystem
 - Great accessibility
- Trade-off has changed, with AI assistance
 - Rust is not that a barrier
 - Stronger compile-time checks
 - Easier to be correct
- Workload has evolved: agentic workloads
 - Long conversations
 - Tool calls & structured outputs
 - UX is not just about being fast, instead, being **reliable**

Design: Layered Boundaries



APIs: OpenAI-compatible HTTP, gRPC, ..

Chat completions: template rendering, reasoning & tool parsing, structured assistant events

Tokenizer & incremental detokenizer

Thin token-in/token-out facade over the engine client

ZMQ transport + MessagePack protocol, request lifecycle & output demux

vllm-tokenizer / vllm-reasoning-parser / vllm-tool-parser

Design: Stream-Native Pipeline

Streaming is the primary design path, not an afterthought.

- Natural: engine produces token outputs over time
- Each layer works as a stream transformation
 - Engine outputs
 - Decoded text delta events
 - Structured chat events
 - SSE chunks
- Incremental, generator-style
- Single source of truth
 - Non-streaming path is just a one-shot & collect

Design: Maintainable Tool Parsers

```
< | DSML | tool_calls>
  < | DSML | invoke name="get_weather">
    < | DSML | parameter name="location" string="true">杭州</ | DSML | parameter>
    < | DSML | parameter name="date" string="true">2024-01-16</ | DSML | parameter>
  </ | DSML | invoke>
  ...
</ | DSML | tool_calls>
```

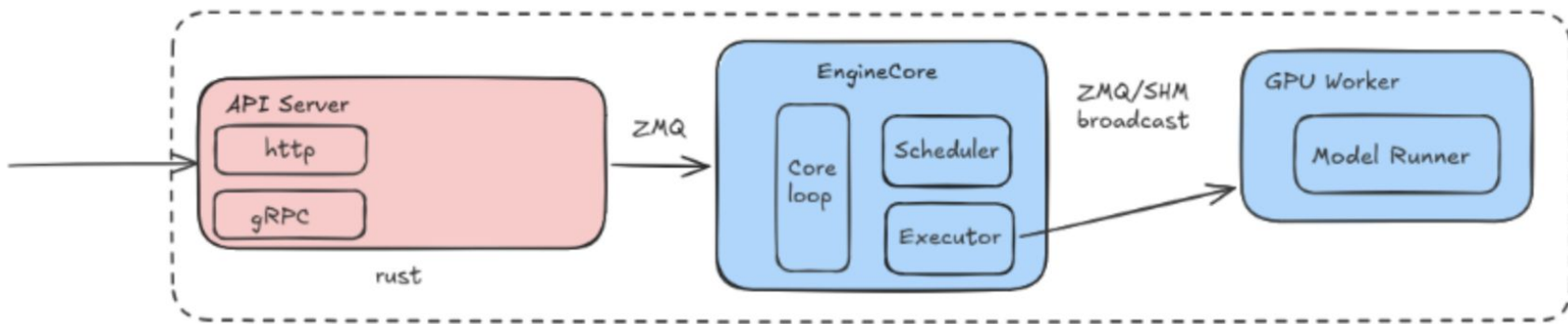
- Existing streaming tool parser impls
 - Mix of regex, ad-hoc string processing, handwritten state machine
- Rebuilt with parser combinators
 - Declarative style
 - Efficient, readable, robust, generable
 - Shared utilities to handle edge cases, like `safe_text`
- Stream-native design ensures robust implementation

Design: Drop-In Integration

- Reuse Python process management for API server multiprocessing
- Controlled by env var `VLLM_USE_RUST_FRONTEND`
- Included in prebuilt vLLM wheels

Alternative: pure Rust entry point

- Replace `vllm serve` with `vllm-rs serve`



Benchmark Results

Setup: Qwen3-0.6B, DP = 4, 4x GB200, inf request rate, concurrency=1024

Decode/streaming-sensitive

- Short input (32), long output (512)
- Prefix cache disabled

Frontend	Throughput (req/s)	P50 TTFT (ms)	P90 TTFT (ms)	P50 TPOT (ms)	P90 TPOT (ms)
Rust	559.79	50.51	67.71	3.29	3.32
Python (default, asc=4)	509.56	165.95	206.52	3.39	3.74
Python (asc=16)	521.80	58.97	80.77	3.54	3.68

Benchmark Results

Setup: Qwen3-0.6B, DP = 4, 4x GB200, inf request rate, concurrency=1024

Preprocess-hot

- Long input (~10K), short output (16)
- Prefix cache fully warmed

Frontend	Throughput (req/s)	P50 TTFT (ms)	P90 TTFT (ms)	P50 TPOT (ms)	P90 TPOT (ms)
Rust	837.00	596.92	807.64	39.90	46.42
Python (default, asc=4)	162.23	6076.09	7936.50	1.96	9.77
Python (asc=32)	785.98	657.15	1211.37	38.89	46.66

Future Direction: Features

Already supported

- Chat completions & completions (streaming & non-streaming)
- Key options & sampling parameters, logprobs, structured outputs
- Renderer, tool / reasoning parser, multimodal support for key models
 - Qwen 3.6, Kimi K2.6, DeepSeek V4, MiniMax M2.7, ...
- Operational: health check, metrics, DP, ...

Feature gaps

- More request parameters: tool_choice, beam search, ...
- APIs: **responses** API, Anthropic **messages** API, ...
- Authentication, trace headers, ...
- More model supports

Future Direction: Gateway / Router

Production vLLM deployments today use extra gateway / router layer

- Routing, load-balancing, P/D disaggregation, KV-aware decisions, ...
 - Even handling frontend work to leave vLLM as just an engine
- `vllm-router` is relatively limited
- People use SMG, Dynamo, ...they are all in Rust!

Rust frontend can become substrate for this work

- Modularized design
- Reusable components
 - Engine protocol
 - Chat rendering / output processing
 - Metrics
- More control-plane work in larger cluster

Closing

Rust frontend:

- Not a fork or a separate inference engine
- Fits into existing vLLM architecture as an alternative frontend
- Not just about performance: for having a better architecture as frontend becomes complex

Integration

- Already merged into main repo!
- Currently still under Inferact: <https://github.com/Inferact/vllm-frontend-rs>
- Integration PR will be merged very soon, behind git submodule: <https://github.com/vllm-project/vllm/pull/40848>

Try it and feedback is welcome!